AI기반 빅데이터 프로젝트

1-3장 상세 학습 보고서

구성

- 1 장: 과목의 학습 흐름, 분석 질문, 재현 가능한 실습 환경과 산출물
- 2 장: 빅데이터의 특성·처리 단계·도구 선택과 생성형 AI 활용의 검증
- 3 장: Netflix 카탈로그 CSV 의 구조·정제·전처리·기초 통계·EDA 해석

전체 개념 연결



1 장. 빅데이터 프로젝트의 전체 구조와 학습 준비

1장 주요 개념



이 장은 한 학기 동안 배우는 도구의 이름을 암기하기보다, 데이터가 수집되어 분석 결과로 쓰이기까지의 흐름과 그 흐름을 반복 실행할 수 있게 만드는 조건을 이해하는 데 초점을 둔다. 이후 Netflix 카탈로그를 분석할 때도 같은 질문을 적용할 수 있다. “데이터는 어디서 왔는가?”, “어떤 변환을 거쳤는가?”, “같은 코드를 다시 돌리면 같은 결과를 얻을 수 있는가?”

1. 학습목표

이 장을 학습한 뒤 다음을 스스로 설명하거나 수행할 수 있어야 한다.

1. 빅데이터 처리, DevOps, 생성형 AI 의 역할을 구별하고 세 요소가 하나의 분석 과제에서 어떻게 만나는지 예로 설명한다.
2. 데이터 파이프라인의 수집·저장·처리·분석·배포를 순서대로 설명하고, 각 단계의 입력·출력과 대표적인 실패를 구별한다.
3. 결과의 *재현 가능성*을 위해 데이터, 코드, 실행 환경, 실행 절차를 함께 기록해야 하는 이유를 설명한다.
4. 생성형 AI 가 작성한 코드나 해석을 검증하는 절차를 제시하고, 검증되지 않은 답변을 제출물에 그대로 넣지 않는다.
5. 로컬 환경에서 도구의 설치 여부와 버전을 확인하고, 문제가 생기면 오류가 발생한 단계를 좁혀 설명한다.

2. 시작하기 전에 알아둘 배경 지식

파일은 바이트의 모음이고, 프로그램은 파일을 읽고 새 파일을 만드는 명령의 집합이다. 예를 들어 CSV를 읽어 결측값을 표시하고 장르별 건수를 집계하면 원본 파일은 '입력', 집계표는 '출력'이다. 입력이 같더라도 실행한 코드 또는 프로그램 버전이 달라지면 결과가 달라질 수 있다. 그러므로 분석 결과만 보관하는 것은 실험 과정을 보존하는 것과 다르다.

3. 왜 세 분야를 함께 배우는가

3.1 빅데이터 처리: 규모와 데이터 흐름의 문제

데이터 분석은 단일 노트북에서 CSV 하나를 읽는 것으로 시작할 수 있다. 그러나 파일 수와 크기가 커지고 갱신 주기가 짧아지면 저장, 전달, 처리에 병목이 생긴다. 빅데이터 기술은 데이터를 여러 컴퓨터에 나누어 보관하거나 계산을 분산하고, 많은 데이터가 계속 들어와도 처리할 수 있도록 설계하는 방법을 다룬다.

3.2 DevOps: 한 번 성공한 분석을 반복 가능하게 만드는 문제

분석가의 컴퓨터에서는 실행되지만 다른 학생의 컴퓨터에서는 실패하는 코드는 공동 프로젝트의 산출물로 부족하다. Python 버전, 설치한 라이브러리, 파일 경로, 환경변수, 실행 순서 중 하나만 달라도 실패할 수 있다. DevOps는 실행 환경을 기술하고 테스트·배포 절차를 반복 가능하게 만드는 데 초점을 둔다. Docker 같은 컨테이너 기술은 실행 환경 차이를 줄이는 수단이다. 다만 컨테이너를 사용했다는 사실만으로 데이터와 코드의 품질이 보장되지는 않는다.

3.3 생성형 AI: 초안 작성 보조이자 검증 대상

생성형 AI는 설명, 코드, 설정 파일의 초안을 빠르게 제안할 수 있다. 그러나 제안한 옵션이 현재 버전에 존재하지 않거나, 데이터의 실제 열을 모른 채 그럴듯한 분석을 만들 수 있다. 따라서 AI의 출력은 결론이 아니라 *검토할 가설 또는

초안*이다. 학습자는 실제 데이터의 값, 실행 결과, 사용 중인 도구의 문서를 대조한 뒤 채택 여부를 결정한다. AI를 사용했더라도 마지막 판단과 제출물의 책임은 사람에게 있다.

세 분야를 하나의 사례로 묶어 보자. 공개 CSV를 읽어 장르별 작품 수를 계산한다고 하자. 데이터를 읽고 결측값을 처리하는 일은 데이터 처리, 누구나 동일한 Python 환경에서 계산하도록 구성하는 일은 DevOps, 집계 코드 초안을 AI에게 요청한 뒤 실제 열 이름과 출력값을 검증하는 일은 AI 활용이다. 어느 한 요소만으로 완성된 분석이 되지 않는다.

> **시험에서 자주 혼동하는 점** "빅데이터 = 반드시 거대한 파일을 쓰는 수업"은 아니다. 작은 데이터셋으로도 파이프라인의 책임, 품질 진단, 재현성 원칙을 먼저 연습할 수 있다. 또한 "AI가 코드를 생성했으니 검증도 AI에게 맡기면 된다"는 주장도 성립하지 않는다. 검증에는 실제 입력 데이터와 실행 환경에 대한 관찰이 필요하다.

4. 데이터 파이프라인의 다섯 단계

파이프라인은 원천 데이터가 분석·활용 가능한 결과가 되기까지의 연결된 과정이다. 다섯 단계는 물리적 서버를 반드시 다섯 대 둔다는 뜻이 아니라 *서로 다른 책임*을 구분한 것이다.

단계	답해야 할 질문	입력과 출력의 예	실패의 예
수집	데이터는 어디에서 어떻게 들어오는가?	공개 데이터 페이지 → 다운로드한 CSV	잘못된 파일을 받거나 일부만 수집함
저장	원본과 가공본을 어디에 보관하는가?	CSV → 원본 보관 영역	원본을 덮어써 변환 전 상태를 잃음
처리	어떤 규칙으로 값을 정제·결합하는가?	원본 CSV → 결측값을 표시한 표	결측을 실제 관측값으로 오인함
분석	어떤 질문에 어떤 지표로 답하는가?	정제된 표 → 장르별 건수와 그래프	작품 수를 시청 인기와 혼동함
배포	다른 사람이 결과를 어떻게 다시 실행하거나 확인하는가?	코드·환경 설명·결과 → 실행 가능한 제출물	작성자 PC에서만 실행됨

단계 사이에는 *인터페이스*가 있다. 예를 들어 처리 코드가 `date_added`를 날짜 문자열로 기대하는데 수집 결과에서 열 이름이 `added_date`로 바뀌면, 두 프로그램이 각자 정상 동작해도 연결 부분에서 실패한다. 따라서 오류가 났을 때는 "전체가 안 된다"라고 말하는 대신 어느 단계의 입력과 출력이 예상과 달랐는지 확인한다.

하나의 도구가 여러 책임을 수행할 수도 있다. 위 표는 도구 이름을 기계적으로 한 칸에 고정하기 위한 분류표가 아니다. **어떤 요구사항을 해결하는지**가 분류 기준이다. 예를 들어 저장소가 동시에 검색과 집계 기능을 제공할 수 있다. 반대로 같은 분석 단계에도 집계표, 시각화, 예측 모델처럼 다른 방법이 있다.

예시 1: 강의실 온도 센서

센서가 매분 온도를 전송한다(수집). 수신한 원본 시각과 온도를 보관한다(저장). 단위가 섞였는지와 비어 있는 관측을 검사한다(처리). 시간대별 평균과 비정상적인 급등 구간을 살핀다(분석). 결과 그래프와 실행 절차를 공유한다(배포). 평균이 이상하게 높다면 분석 수식뿐 아니라 센서 단위와 수집 시각부터 되짚어야 한다.

예시 2: Netflix 작품 카탈로그

공개 CSV를 입수한다(수집). 원본을 변경하지 않고 별도로 보관한다(저장). `date_added`를 날짜로 해석하고 `listed_in`의 여러 장르를 구분한다(처리). 유형별 작품 수와 연도별 등록 건수를 계산한다(분석). 다른 학생이 같은 코드와 환경에서 표를 재생성하도록 한다(배포). 중요한 한계는 카탈로그의 한 행이 *작품 정보*라는 점이다. 이 데이터만으로 특정 작품을 몇 명이 시청했는지는 알 수 없다.

5. 평가와 산출물은 어떻게 연결되는가

1장 원문은 프로젝트, 중간·기말시험, 과제, 출석, 참여도를 평가의 구성 요소로 설명하고, 여러 과제를 하나의 프로젝트에 누적하는 구조를 제시한다. **정확한 시험 범위·일정·비중은 해당 학기의 최신 강의 공지를 우선**한다. 시험 준비의 핵심은 날짜를 외우는 것이 아니라 산출물 사이의 의존 관계를 이해하는 것이다.

첫 실습에서 만든 환경 설명과 저장소 구조는 이후 코드와 실험 결과를 보관하는 기반이다. 데이터 처리 과제에서 작성한 코드와 실행 로그는 후속 비교 실험의 기준이 된다. 비교 결과를 설명할 때는 입력 데이터, 실행 조건, 측정 방법이 함께 있어야 한다. 나중에 AI 보조 분석을 활용한다면 어떤 제안을 받아 어떤 부분을 채택했고 어떻게 확인했는지 기록해야 한다. 최종 보고서는 이 기록을 모아 방법·결과·한계를 독자가 확인할 수 있게 만드는 문서다.

예를 들어 두 처리 방식의 실행 시간을 비교하면서 A는 100행, B는 100만 행을 사용했다면 숫자만 나열해서는 공정한 비교가 아니다. 반대로 같은 입력과 조건을 기록하면 빠른 방식뿐 아니라 왜 차이가 났는지 토론할 수 있다. 이런 이유로 작은 환경 보고서와 실행 로그가 뒤의 분석 결과만큼 중요하다.

꼭 구분해야 할 개념

혼동하기 쉬운 쌍	구별하는 질문
원본 데이터와 정제 데이터	나중에 정제 규칙을 바꾸면 원본부터 다시 만들 수 있는가?
프로그램 설치와 실행 성공	버전 문자열이 출력되는 것과 실제 작업이 완료되는 것은 같은가?
분석 지표와 해석	그래프가 보여 준 값과 그 값의 원인에 대한 주장을 구분했는가?
AI가 제안한 코드와 검증된 코드	실제 데이터·환경에서 실행하고 결과를 확인했는가?
저장소에 있는 코드와 재현 가능한 제출물	입력 데이터 취득 방법·환경·실행 순서가 함께 설명되어 있는가?

핵심 요약

- 빅데이터 처리는 데이터 규모와 흐름을, DevOps 는 반복 실행 가능한 환경과 배포를, 생성형 AI 는 초안 작성의 보조를 다룬다. AI 출력은 검증 대상이다.
- 파이프라인의 수집·저장·처리·분석·배포는 제품 목록이 아니라 책임의 구분이다. 단계의 입력·출력과 실패 지점을 함께 설명해야 한다.
- 재현 가능한 분석에는 결과 그림만이 아니라 데이터, 코드, 버전, 실행 절차, 판단 근거가 필요하다.
- Netflix 같은 공개 카탈로그의 작품 수는 시청 수나 인기의 직접 지표가 아니다. 관찰 단위를 확인한 후 질문을 만들어야 한다.
- 환경 점검에서 가장 중요한 결과는 “모두 최신”이라는 선언이 아니라 *실제로 관찰한 버전과 문제*를 기록하는 것이다.

2 장. 빅데이터 개론과 생성형 AI 활용

2장 주요 개념



> 학습 범위: 2주차 교안의 5V, 처리 규모 판단, 데이터 파이프라인, 운영체제 기초, 생성형 AI 초안 검증. 이 장은 HDFS의 구체적인 설치·명령이나 Titanic EDA를 다루지 않는다. 아래 사례에서 따로 밝힌 수치는 학습용 가정이며 실제 시스템 측정값이 아니다.

1. 이 장을 공부한 뒤 할 수 있어야 하는 일

1. 주어진 데이터의 **Volume·Velocity·Variety·Veracity·Value** 를 각각 측정 가능한 질문으로 바꾼다.

2. 단일 컴퓨터의 처리 한계가 무엇인지 먼저 관찰한 다음 스케일업과 스케일아웃 중 적절한 방향을 근거와 함께 설명한다.
3. 수집→저장→처리→분석→운영 계층에서 주요 도구가 맡는 역할을 구분하고, 하나의 도구가 모든 문제를 해결한다고 오해하지 않는다.
4. 프로세스·CPU·메모리·파일시스템·네트워크 자원이 데이터 작업의 시간과 실패에 미치는 영향을 설명한다.
5. 생성형 AI 의 설계·코드 초안에서 존재하지 않는 API, 버전 불일치, 실행 불가능한 연산, 출처 없는 성능 수치를 찾아 검증 기록을 작성한다.

2. 선수 지식: 이 용어부터 확인하기

- **행(row)과 열(column):** 표의 한 행은 관찰 대상 한 건이고 열은 그 대상의 속성이다. 웹 로그 한 행이 요청 한 건이라면 `status` 열은 응답 상태코드일 수 있다. 행 수와 파일 크기는 다르며, 같은 행 수라도 문자열 길이에 따라 메모리 사용량이 달라진다.
- **저장장치와 메모리:** SSD 의 파일 크기와 pandas 가 읽어 RAM 에 펼친 크기는 같지 않다. 문자열 객체나 자료형 표현이 추가 공간을 차지할 수 있다.
- **처리량과 지연:** 처리량은 단위 시간에 처리한 건수(예: 건/초), 지연은 한 건이 들어와 결과가 나오기까지 걸린 시간이다. 유입이 처리보다 빠르면 대기열이 자란다.
- **배치와 스트리밍:** 배치는 일정 기간 자료를 모아 처리한다. 스트리밍은 계속 들어오는 이벤트를 지속적으로 처리한다. 실시간이라는 표현만으로 처리 지연의 허용 범위를 알 수 없으므로 “5 분 이내”처럼 수치로 정한다.
- **병목:** 전체 시간을 가장 강하게 제한하는 자원이다. CPU 사용률이 낮더라도 디스크 읽기나 네트워크 대기 때문에 작업이 느릴 수 있다.

3. 빅데이터를 보는 다섯 가지 질문: 5V

빅데이터를 “몇 GB 이상”이라고 고정된 숫자로 정의하기보다, **지금 가진 장비와 필요한 결과의 시간·품질을 함께** 고려한다. 작은 데이터라도 초당 빠르게 들어오면 수집 문제가 될 수 있고, 큰 데이터라도 한 번 순차 읽으면 단일 장비에서 충분할 수 있다. 5V는 제품 이름이 아니라 문제를 진단하기 위한 질문 묶음이다.

축	학생이 던질 질문	관찰·계산의 예	판단에서 주의할 점
Volume(양)	얼마나 쌓이고 얼마만큼 읽는가?	현재 300 GB, 하루 12 GB 증가	디스크 용량뿐 아니라 분석 때 필요한 RAM·읽기 시간 확인
Velocity(속도)	얼마나 빨리 들어오고 언제 결과가 필요한가?	초당 400건, 5분 내 집계	초당 건수와 허용 지연을 분리해서 측정

축	학생이 던질 질문	관찰·계산의 예	판단에서 주의할 점
Variety(형태)	입력 구조가 몇 종류이고 자주 바뀌는가?	CSV 2종과 JSON 1종	파일 형식뿐 아니라 열 이름·자료형 변화도 확인
Veracity(신뢰성)	빠졌거나 잘못되거나 중복된 건은?	필수 열 결측률 2%, 중복률 1%	"정제했다"가 아니라 처리 전후 건수를 남김
Value(가치)	이 결과로 어떤 결정을 더 잘 하는가?	장애 탐지 시간 단축	데이터 크기만으로 가치가 생기지 않음

사례 1: 같은 100 GB 라도 기술 선택이 달라지는 이유

연구실 A는 100 GB CSV를 한 달에 한 번 읽어 보고서를 만든다. 연구실 B는 총량은 100 GB지만 로그가 계속 유입되어 2분 안에 장애 알림이 필요하다. A의 우선 질문은 "단일 컴퓨터가 정해진 마감 전 읽고 집계할 수 있는가?"다. B의 우선 질문은 "들어오는 속도보다 처리 속도가 느려 대기열이 증가하는가?"다. 따라서 A에는 먼저 단일 컴퓨터의 실제 실행 시간·메모리를 재고, B에는 유입률·처리률·지연을 재야 한다. **총량이 같다고 같은 기술을 택하지 않는다.**

사례 2: 5V 를 수치로 읽는 연습

다음은 학습용 가정이다. 서비스가 하루 12 GB의 로그를 만들고 30일 보존하며, 초당 평균 400건·최대 1,200건이 들어온다. 현재 수집 프로그램의 최대 처리량이 초당 900건이라면 평상시에는 처리할 수 있지만 최대 유입 구간에는 매초 약 `1,200 − 900 = 300`건씩 대기열이 늘어난다. 최대 구간이 10분 계속되면 `300 × 600 = 180,000`건이 쌓인다. 저장량도 단순히 `12 × 30 = 360 GB`만 고려하면 안 된다. 임시 파일, 인덱스, 복제 또는 백업 방식에 따라 추가 공간이 필요하다. 여기서 우선 해결할 문제는 최고 속도 구간의 적체와 보존 비용이지, "360 GB니까 무조건 분산 처리"라는 결론이 아니다.

적용 한계: 평균 유입률 만으로 피크를 판단할 수 없다. 로그의 크기 분포, 압축률, 처리 방식, 허용 지연이 바뀌면 계산도 바뀐다.

4. 스케일업과 스케일아웃: 어떤 병목을 어떻게 풀 것인가

스케일업은 한 컴퓨터에 CPU·RAM·빠른 저장장치를 더하는 방식이다. 기존 단일 컴퓨터 프로그램을 유지하기 쉽지만 장비의 최대 사양과 가격, 한 장비 장애에 대한 제약이 있다. **스케일아웃**은 컴퓨터 여러 대에 작업과 데이터를 나누는 방식이다. 더 많은 자원을 확보할 수 있지만 분배·통신·재시도 비용과 부분 장애를 설계해야 한다. 여러 대라는 이유만으로 작업이 N배 빨라지지 않는다.

예를 들어 모든 행을 읽은 뒤 `status`별 건수를 세는 작업에서 한 컴퓨터의 RAM이 부족하면 디스크를 사용하거나 분할 읽기·더 큰 RAM·여러 컴퓨터로의 분산을 검토할 수 있다. 먼저 필요한 것은 **문제가 RAM 부족인지, CPU 포화인지, 디스**

크 읽기인지 확인하는 일이다. RAM 부족을 네트워크 증설로 해결할 수는 없다.

사례 3: 속도 비교를 어떻게 해석할까

원문 교안에는 학습용 *예시*로 1 GB 작업이 단일 노드와 8노드 모두 1분, 10 GB에서는 각각 9분과 2분, 100 GB에서는 각각 88분과 13분이라고 제시한다. 10 GB의 속도비는 `9 ÷ 2 = 4.5`배, 100 GB는 약 `88 ÷ 13 ≈ 6.8`배다. 그런데 8노드인데 8배가 아닌 이유는 작업 제출·노드 간 통신·데이터 재배치(셔플)와 병렬화할 수 없는 부분이 있기 때문이다. 1 GB에서는 이러한 고정 비용이 이득을 상쇄한다. **이 수치는 실제 연구실 장비의 측정 결과가 아니다.** 현실에서는 같은 데이터와 같은 집계, 같은 환경에서 여러 차례 실행해 비교해야 한다.

판단 순서: (1) 결과의 정확성을 먼저 맞춘다 → (2) 데이터 크기와 실행 시간을 잰다 → (3) CPU·RAM·디스크·네트워크 중 병목을 확인한다 → (4) 마감 시간과 비용을 비교한다 → (5) 단일 노드 개선으로 해결되지 않을 때 분산 실행을 검토한다.

오개념: "100 GB 이상이면 언제나 분산 시스템이 필요하다." → 장비·파일 형식·연산·마감 시간에 따라 다르다. "노드 8대면 8배 빨라진다." → 고정 비용과 통신 비용 때문에 일반적으로 성립하지 않는다.

5. 파이프라인의 다섯 계층과 운영체제의 역할

웹 로그를 서비스 운영 대시보드로 만든다고 가정하자. **수집**은 웹 서버에서 로그를 받아들이는 일(Kafka, Filebeat·Logstash 등), **저장**은 받은 자료를 유지하는 일(HDFS 또는 객체 저장소 등), **처리**는 로그를 정리·변환·집계하는 일(MapReduce·Spark 등), **분석**은 상태코드 분포를 읽거나 검색·시각화하는 일(예: Elasticsearch·Kibana), **운영**은 전체 실행 환경을 일관되게 배포·감시하는 일(Docker·Kubernetes·CI/CD 등)이다. 제품 목록을 모두 외우기보다 "어떤 데이터가 어디서 들어와 어떤 결과가 나오나"를 말할 수 있어야 한다. 실습 단계에서는 한 컴퓨터에서 일부 계층만 구현해도 원리는 동일하다.

운영체제는 이 도구들 아래에서 실제 자원을 제공한다. **프로세스**는 실행 중인 프로그램이다. CPU가 여러 프로세스의 실행 시간을 나눠 주고, **메모리**는 현재 계산에 필요한 데이터를 담으며, **파일시스템**은 저장된 파일을 이름과 위치로 관리하고, **네트워크**는 다른 컴퓨터와 데이터를 주고받는다. 처리 코드만 보고 "Spark가 느리다"고 단정할 수 없다. 디스크에서 입력을 기다리는지, 메모리가 부족해 중간 데이터를 디스크에 기록하는지, 네트워크 전송이 긴지 확인해야 한다.

관찰 예시: 명령의 의미

원문은 리눅스에서 `ps`, `free`, `df`로 프로세스·메모리·디스크 공간을 확인한다. `ps`의 높은 CPU 사용률은 계산 병목의 단서일 수 있으나 한 번의 수치만으로 결론을 내리지 않는다. `free`에서 메모리 여유가 적으면 메모리 압박을 의심하고, `df`에서 여유 공간이 거의 없으면 결과 파일 저장 실패를 의심한다. 이 명령은 **진단의 시작점**이지 원인 확정 도구가 아니다.

적용 한계: HDFS 블록·복제, Spark 실행자 내부 구조 등은 뒤 주차의 학습 범위다. 이번 장에서는 저장·처리·운영의 역할

과 자원 병목의 연결만 이해한다.

6. 생성형 AI: 초안은 도움을 주지만 증거는 아니다

생성형 AI는 요구사항을 정리하거나 설계 초안·설정 예시·오류 설명을 제안할 수 있다. 하지만 실제로 없는 함수 이름을 만들거나, 버전이 맞지 않는 라이브러리를 섞거나, 측정하지 않은 성능을 숫자로 말할 수 있다. 따라서 **제안 → 확인 → 실행 → 기록**의 순서를 지킨다.

원문 교안의 검증 전 예시는 존재하지 않는 `auto_schema` 함수와 `KafkaStreamReader` 클래스를 제시하고, Spark 3.5.x 실행 환경에 2.4.0용 커넥터 좌표를 섞으며, 근거 없이 "1 TB를 8노드에서 30초"라고 쓴다. 학생은 이 코드를 복사해 실행하는 대신 (1) 해당 모듈을 import할 수 있는지, (2) 현재 런타임 버전과 의존성 버전이 맞는지, (3) 간단한 입력으로 실제 실행되는지, (4) 속도 숫자는 어떤 조건에서 재었는지를 확인해야 한다. 검증에서 실패하면 **무엇이 실패했는지·어떤 증거가 나왔는지·어떻게 수정할지**를 함께 기록한다.

사례 4: "그럴듯한 코드"를 검사하는 질문

AI가 `from pyspark.sql.functions import auto_schema`를 제안했다면 우선 사용 중인 환경에서 import를 실행하거나 해당 버전의 공식 API 목록을 확인한다. import 실패는 그 함수가 현재 환경에서 사용 가능하지 않다는 직접 증거다. 이에 비해 "예전에 다른 강의에서 본 것 같다"는 검증 근거가 아니다. AI가 처리량을 단언하면 데이터 크기·장비·파일 형식·실행 횟수·측정 로그가 없는 수치는 채택하지 않는다.

오개념: "AI가 정리한 도구 비교표는 최신일 것이다." → 모델의 학습 시점과 실제 버전이 다를 수 있다. "코드가 실행됐으니 결과도 정확하다." → 실행 성공과 분석 질문·집계 단위·결측 처리의 타당성은 별개다.

핵심 정리

6. 5V 는 정해진 용량 기준이 아니라, 양·속도·형태·품질·가치를 각각 관찰하도록 돕는 틀이다.
7. 단일 컴퓨터와 분산 시스템의 선택은 데이터 크기만이 아니라 마감 시간, 병목, 비용, 운영 부담에 달려 있다.
8. 분산 실행에는 시작·통신·재배치 비용이 있어서 작은 작업은 더 느릴 수도 있다.
9. 수집·저장·처리·분석·운영은 다른 역할이며, 운영체제의 CPU·메모리·파일시스템·네트워크 자원이 모두 이 역할의 실행 기반이다.
10. 생성형 AI 의 답변은 검증 전 초안이다. 함수·버전·실행 결과·성능 주장에 각각 확인 근거를 붙인다.

3 장. Netflix 카탈로그 데이터의 정제와 탐색적 분석

3장 주요 개념



상세 학습목표

이 장을 학습한 뒤에는 (1) Kaggle의 Dataset과 Code/Notebook의 역할을 구분하고 분석 질문에 맞는 파일을 선택할 수 있다. (2) 한 행의 관찰 단위와 12개 열의 의미를 설명하고, 실제 작품 세 건에서 각 값이 같은 행에 속함을 읽어 낼 수 있다. (3) 결측·중복·형식 오류를 먼저 진단하고, 원본을 보존하면서 정제와 전처리를 구분해 적용할 수 있다. (4) 콘텐츠 유형, 상영시간, 등록 연도, 장르·등급 조합을 질문에 맞는 분모와 시각화로 탐색할 수 있다. (5) 카탈로그 메타데이터에

서 말할 수 있는 사실과 시청·선호·인과관계처럼 말할 수 없는 주장을 구분할 수 있다.

공부 전 알아둘 배경지식

- **행과 열:** 표의 한 행은 하나의 관찰 대상, 열은 그 대상의 속성이다. 이 파일은 이용자별 시청 기록이 아니라 영화 또는 TV 프로그램 **한 건**이 한 행이다.
- **범주형과 수치형:** `type`(Movie/TV Show), `rating`(시청 등급)은 범주형이다. `release_year` 는 숫자로 저장돼도 우선 시간의 위치를 뜻한다. `duration` 은 원래 문자열이며 영화의 분과 TV 의 시즌을 분리해야 수치 계산이 가능하다.
- **빈도와 비율:** 빈도는 몇 건인지, 비율은 어떤 전체를 분모로 삼아 나눈 값인지까지 함께 말해야 한다. 예컨대 5,377 편을 전체 7,787 건으로 나눈 69.1%는 '카탈로그 내 영화 비율'이지 '영화 시청률'이 아니다.
- **평균과 중앙값:** 평균은 모든 값을 합쳐 건수로 나눈 값이고 긴 작품 같은 극단값의 영향을 받는다. 중앙값은 값을 정렬했을 때 가운데 값이다. 사분위수 Q1·Q3 는 하위 25%·75% 지점이며, Q3−Q1 은 가운데 절반의 폭이다.

3.1 Kaggle 에서 데이터를 고르고 질문을 제한하기

Kaggle의 **Dataset**은 파일과 설명을, **Code/Notebook**은 제공자들의 실행 코드를 제공한다. Competition은 정해진 문제와 평가 방식을, Discussion은 질문과 의견을 제공한다. 타인의 Notebook은 분석 아이디어를 참고할 수 있지만 다른 시점의 파일을 사용했을 수 있다. 따라서 이름이 같은 Netflix 파일이라도 행 수와 열이 같은지 확인하고 자기 파일에서 결과를 다시 실행한다.

분석 목적에 맞는 파일인지 판단하는 가장 빠른 질문은 "한 행이 무엇이며, 내가 묻는 현상이 어느 열에 기록돼 있는가?"이다. 수업 파일은 작품의 유형, 제작 연도, 등록일, 시청 등급, 장르, 상영시간 등을 포함한다. 따라서 **현재 파일에 남아 있는 작품의 구성과 메타데이터 품질**은 분석할 수 있다. 그러나 재생 수, 이용자, 매출, 제작비 열이 없으므로 **최다 시청 작품, 선호 장르, 투자수익률**은 이 파일만으로 계산할 수 없다.

예시 질문을 비교해 보자.

질문	이 파일로 가능한가	이유 및 필요한 추가 자료
Movie와 TV Show는 각각 몇 건인가?	가능	`type`을 세면 된다.
어떤 장르의 **보유 작품 수**가 많은가?	가능	`listed_in`의 복수 장르를 분리해 고유 작품 수를 센다.
가장 **인기 있는** 장르는 무엇인가?	불가능	작품 보유량과 시청·선호는 다르다. 작품별 재생 로그·고유 시청자 수·시청 시간 등이 필요하다.

질문	이 파일로 가능한가	이유 및 필요한 추가 자료
등록 건수가 늘어 가입자가 늘었는가?	불가능	가입·해지 시계열, 가격·광고 등 다른 요인과 인과분석 설계가 필요하다. 등록 건수의 동시 증가만으로 원인을 증명할 수 없다.
어떤 작품을 추천해야 하는가?	불가능	사용자별 시청·검색·평가 이력과 검증 자료가 필요하다.

시험 포인트: 분석 질문을 쓸 때에는 그 답에 필요한 열을 직접 짚어 보아야 한다. `rating`은 사용자 별점이 아니라 **시청 등급**이며, `date_added`는 제작일이나 이용자의 시청일이 아니라 **Netflix 카탈로그 등록일**이다.

3.2 실제 행으로 관찰 단위와 열 의미 익히기

수업 파일은 **7,787행 × 12열**이다. `show_id`는 작품 식별자 후보이며 수업 고정본에서 결측·중복이 없다. 첫 세 행의 값은 다음과 같다. `3%`, `7:19`, `23:59`는 퍼센트나 시각 계산값이 아니라 **실제로 `title` 열에 기록된 작품명**이다.

show_id	type	title	country	release_year	rating	duration
s1	TV Show	3%	Brazil	2020	TV-MA	4 Seasons
s2	Movie	7:19	Mexico	2016	TV-MA	93 min
s3	Movie	23:59	Singapore	2011	R	78 min

예를 들어 s2는 **제목이 7:19인 영화**이고 상영시간은 **93분**이다. s1의 감독은 비어 있고, s2의 감독은 Jorge Michel Grau, s3의 감독은 Gilbert Chan이다. 이처럼 한 행의 여러 열을 같이 읽으면 제목을 잘못된 단위로 해석하는 오류를 줄일 수 있다.

나머지 열까지 포함한 의미를 정리하면 다음과 같다.

열	실제 의미	해석에서 피할 오류
`show_id`	작품 식별자	제목이 같다는 이유만으로 중복 작품이라 단정하지 않는다.
`type`, `title`	영화/TV 유형, 작품명	`title` 문자열을 길이나 시청 시간으로 읽지 않는다.
`director`, `cast`	감독, 출연진	비어 있을 수 있고 한 셀에 여러 명이 기록될 수 있다.

열	실제 의미	해석에서 피할 오류
`country`	제작 국가	공동 제작이면 한 작품에 국가가 둘 이상이다.
`date_added`, `release_year`	Netflix 등록일, 최초 공개 연도	서로 다른 사건의 날짜다.
`rating`	시청 등급	별점·만족도가 아니다.
`duration`	영화는 분, TV는 시즌	숫자만 뽑아 전체 평균을 내지 않는다.
`listed_in`	쉼표로 구분된 복수 장르	한 작품이 여러 장르에 속한다.
`description`	작품 소개문	실제 시청 후기나 반응이 아니다.

이 표는 특정 시점에 남아 있는 카탈로그의 **스냅샷**이다. 역사상 등록됐다가 사라진 콘텐츠는 포함되지 않을 수 있으며, 이용자가 무작위로 뽑힌 표본도 아니다. "Netflix의 모든 시점 모든 작품"으로 일반화하는 말은 피한다.

3.3 원본 확인과 데이터 품질 진단

실습의 첫 단계는 파일을 바로 정제하는 것이 아니라 **어떤 파일을 읽었고 무엇이 누락됐는지** 알아보는 것이다. Conda 실습 폴더의 `data/raw/netflix_titles.csv`를 입력으로 두고 원본 파일은 덮어쓰지 않는다. 예제 결과가 수업 숫자와 다르면 7,787행·12열 및 열 이름을 먼저 확인한다.

```
from pathlib import Path
import pandas as pd


raw = Path("data/raw/netflix_titles.csv")
df = pd.read_csv(raw, dtype={"show_id": "string"})
print(df.shape)                                  # (7787, 12)
print(df.head(3)[["show_id", "type", "title", "duration"]])
print("ID 결측:", df["show_id"].isna().sum())
print("ID 중복:", df["show_id"].duplicated().sum())


missing = pd.DataFrame({
    "missing_n": df.isna().sum(),
    "missing_pct": (df.isna().mean() * 100).round(1),
}).sort_values("missing_n", ascending=False)
print(missing)
```

수업 자료에서 `director`는 **2,389건**, `cast`는 **718건**, `country`는 **507건**이 누락됐다. `director`의 결측률을 직접 계산하면 `2389 ÷ 7787 × 100 ≈ 30.7%`다. 전체 행을 감독이 비었다는 이유만으로 버리면 다른 분석에도 쓸 수 있는 작품을 크게 잃는다. 감독 분석에는 감독 정보가 있는 행만, 유형 분석에는 전체 행을 쓰는 식으로 **질문별 유효 N**을 분리한다. 공백 문자열 `"  "`은 일반적인 `isna()`에서 누락으로 잡히지 않을 수 있으므로 문자열의 앞뒤 공백도 살펴야 한다.

중복 검사도 기준이 중요하다. `show_id` 중복은 식별자 계약 위반이지만, 같은 제목의 작품은 동명 작품이나 재공개일

수 있다. `df.duplicated(subset=["title"])`가 양수라고 자동으로 삭제하면 안 된다. 이상값도 마찬가지다. 300분을 넘는 영화가 있다면 긴 정상 작품인지 입력 오류인지 조사한 뒤 결정한다.

3.4 정제와 전처리: 원본을 바꾸지 않고 분석용 표 만들기

정제는 공백·표기·형식 문제를 고치는 일이다. **전처리**는 날짜를 연·월로 분해하거나 다중 장르를 관계형 표로 펼쳐 특정 분석이 가능하도록 만드는 일이다. **EDA**는 그렇게 얻은 표를 이용해 분포와 관계를 탐색하는 일이다. 순서는 "진단 → 처리 규칙 → 변환 → 변환 후 검증"이다.

결측을 `Unknown`으로 채우면 결측 자체가 사라지는 것이 아니라 '미상'이라는 범주가 생긴다. 예를 들어 감독 미상을 감독 순위에 끼워 넣으면 실제 감독처럼 보인다. 목적에 맞게 별도로 표시하거나 감독 순위에서 제외하되, 제외 건수는 기록한다.

다음 짧은 코드는 수업 제공 `netflix_pipeline.py`의 `prepare`와 `normalize_multi_values`를 이용한다. 실습 폴더에서 실행하며, 함수는 원본 `df`를 바꾸지 않고 분석용 `clean`과 `genres`를 만든다.

```
from netflix_pipeline import prepare, normalize_multi_values


clean = prepare(df)
genres = normalize_multi_values(clean, "listed_in")
print(clean[["type", "duration", "duration_value", "duration_unit"]].head(3))
print(len(genres), genres["show_id"].nunique())
```

예상 첫 행은 `TV Show`, `4 Seasons`, 값 4, 단위 `Seasons`이고, 둘째 행은 `Movie`, `93 min`, 값 93, 단위 `min`이다. 영화 93분과 TV 4시즌을 평균 내는 것은 **단위가 다른 양의 합산**이므로 의미가 없다. `date_added`에서 만든 `added_year`도 최초 공개 연도인 `release_year`와 구별한다.

`listed_in`은 한 셀에 여러 장르가 쉼표로 들어 있다. 이를 한 작품-한 장르씩 펼친 표가 `genres`다. 수업 고정본에서는 **7,787개 작품에서 17,071개 콘텐츠-장르 관계**가 나온다. 관계 수가 작품 수보다 큰 것은 작품 하나가 여러 장르에 속하기 때문이다. 이 원리는 출연진·감독·국가에도 적용된다. 원 교안의 결과는 감독 관계 6,114건(5,398작품), 출연진 관계 55,953건(7,069작품), 국가 관계 9,063건(7,280작품)이다.

3.5 단변량 EDA: 한 변수의 분포를 읽기

단변량 분석은 하나의 열이 어떤 값을 얼마나 갖는지 묻는다. 범주형에는 빈도표와 막대그래프, 수치형에는 중앙값·사분위수와 히스토그램·상자그림을 주로 쓴다. 장르처럼 여러 범주가 한 작품에 중첩되는 경우는 일반적인 단일 범주 구성비와 다르게 읽어야 한다.

```
type_n = clean["type"].value_counts()
type_pct = (type_n / len(clean) * 100).round(1)
print(pd.DataFrame({"건수": type_n, "전체 대비 %": type_pct}))


ax = type_n.plot.bar(color=["#27638b", "#8caabe"], rot=0)
ax.set(xlabel="콘텐츠 유형", ylabel="카탈로그 건수", title="유형별 작품 수")
```

기대 결과는 **Movie 5,377건(69.1%)**, **TV Show 2,410건(30.9%)**이다. 이 분모는 카탈로그의 7,787건이다. 영화가 더 많이 **등재**돼 있다는 뜻이지, 영화가 더 많이 **시청**됐다는 뜻은 아니다.

영화 상영시간의 분포는 TV 시즌 수와 별도로 본다. 수업 고정본의 **5,377편 영화**에서 평균은 약 **99.3분**, 중앙값은 **98분**, Q1은 **86분**, Q3는 **114분**이다. 가운데 절반의 길이는 86-114분 범위이고 사분위범위는 **28분**이다. 최솟값 3분과 최댓값 312분은 극단적이지만, 상자그림에서 멀리 찍힌다는 이유만으로 삭제하지 않는다.

```
import matplotlib.pyplot as plt


movies = clean.loc[
    clean["type"].eq("Movie") & clean["duration_unit"].eq("min"),
    "duration_value",
].dropna().astype(float)
print(movies.agg(["count", "mean", "median", "min", "max"]))
print(movies.quantile([0.25, 0.75]))
movies.plot.hist(bins=30, title="영화 상영시간 분포")
plt.xlabel("분")
plt.tight_layout()
plt.show()
```

막대그래프에서는 빈도 비교를 왜곡하지 않도록 0에서 시작하는 세로축을 사용한다. 히스토그램은 구간 폭을 바꾸면 모양이 달라질 수 있으므로 축과 구간 수를 함께 보아야 한다.

3.6 이변량 EDA: 두 변수의 관계와 올바른 분모

두 범주형 변수 `listed_in`(장르)과 `rating`(시청 등급)을 함께 보면 조합별 작품 관계 수를 알 수 있다. 장르별로 전체 관계 수가 다르면 단순 건수표에서 큰 장르가 두드러진다. **각 장르 안에서 어떤 등급이 차지하는 비율**을 묻는다면 그 장르의 관계 수로 각 칸을 나눈 행 비율표를 만들어야 한다. 한 작품이 복수 장르에 속할 수 있으므로 이 교차표의 관찰 단위는 독립된 이용자나 고유 작품이 아니라 **작품-장르 관계**다.

```
genre_rating = genres.merge(clean[["show_id", "rating"]], on="show_id", how="left")
table = pd.crosstab(
    genre_rating["value"], genre_rating["rating"].fillna("Missing")
)
top = table.loc[table.sum(axis=1).nlargest(5).index]
row_pct = top.div(top.sum(axis=1), axis=0).mul(100).round(1)
print(top)
print(row_pct)
```

`top`의 한 칸이 200이면 “그 장르-등급 조합의 관계가 200건”이라는 뜻이다. `row_pct`의 한 칸이 40이라면 “그 장르의 관계 중 해당 등급이 40%”라는 뜻이다. 분모가 달라 건수표와 비율표는 다른 질문에 답한다. 등급 누락을 조용히 버리지 않으려고 `Missing`으로 표시하되, 이는 실제 시청 등급이 아니다.

범주형과 수치형을 함께 볼 때는 Movie만 고른 뒤 등급별 영화 길이를 비교한다. 그룹별 유효 N, 중앙값, 사분위범위를 함께 보아야 하며, 한 그룹의 표본이 매우 적거나 상자가 크게 겹치면 차이를 과장하지 않는다. 등급별 길이의 차이가 관찰돼도 **시청 등급이 길이를 변화시켰다**는 인과관계는 이 표만으로 증명할 수 없다.

3.7 시간과 다중값 EDA 의 해석

`release_year`는 작품이 처음 공개된 연도, `date_added`는 Netflix에 등록된 날이다. 2010년 공개 영화가 2019년에 등록되는 것은 자연스럽다. `added_year - release_year`는 두 시점의 간격이지만 음수가 나왔다고 무조건 계산 오류로 삭제하지 않는다. 수업 고정본에서 음수 후보가 **12건** 있으므로 날짜 정의와 원문을 먼저 재확인한다.

```
annual = (
    clean.dropna(subset=["added_year"])
    .groupby("added_year")
    .size()
    .rename("카탈로그에 남아 있는 등록 건수")
)
print(annual.loc[2017:2021])
annual.plot.line(marker="o", title="현재 스냅샷의 등록 연도별 작품 수")
plt.xlabel("Netflix 등록 연도")
plt.ylabel("건수")
plt.tight_layout()
plt.show()
```

수업 원고의 연도별 값은 **2017년 1,225건, 2018년 1,685건, 2019년 2,153건, 2020년 2,009건**이다. **2021년은 117건**이지만 부분 연도이므로 2020년 전체와 바로 비교해 "등록 전략이 축소됐다"고 말하면 안 된다. 더구나 현재 스냅샷에서 이미 삭제된 과거 콘텐츠는 빠질 수 있다. 이 선은 완전한 역사적 입출고 로그가 아니라 **현재 남아 있는 콘텐츠의 등록일 분포**다.

장르 빈도에서는 `genres["value"].value_counts()`가 작품-장르 관계 수를 센다. 수업 고정본의 상위 장르는 **International Movies 2,437건, Dramas 2,106건, Comedies 1,471건**이다. 이는 카탈로그에 이 장르로 분류된 작품이 몇 편 있는지를 나타내며, "시청자가 선호한다"는 뜻은 아니다. 여러 장르에 속한 작품은 각 장르에서 각각 한 번씩 세므로 장르별 보유 비율을 모두 더하면 100%를 넘을 수 있다.

3.8 시험에서 자주 헷갈리는 문장 고치기

잘못된 문장	데이터가 뒷받침하는 문장
"`rating`이 높은 작품이 인기가 많다."	"`rating`은 관람 등급이며 선호나 평점 열이 아니다."
"International Movies가 가장 인기 있는 장르다."	"이 스냅샷에서 International Movies로 분류된 작품-장르 관계가 2,437건이다."
"2021년에 Netflix 등록이 급감했다."	"수업 파일에서 2021년 등록일이 기록된 콘텐츠가 117건이며, 관측 기간이 짧아 완전 연도와 직접 비교할 수 없다."
"영화와 TV 평균 duration은 50이다."	"영화의 분과 TV의 시즌 수는 단위가 달라 함께 평균 낼 수 없다."

잘못된 문장	데이터가 뒷받침하는 문장
“감독 결측 행은 모두 분석에서 제거한다.”	“감독을 묻는 분석에서만 결측 처리 후 유효 N을 밝힌다.”
“장르 관계가 17,071건이므로 작품도 17,071편이다.”	“작품은 7,787건이고 한 작품의 복수 장르를 펼쳐 관계가 17,071건이다.”

핵심 요약

Netflix 고정본의 관찰 단위는 **카탈로그 작품 한 건**이고 7,787행·12열이다. 분석은 “질문 → 관찰 단위·필요 열 → 품질 진단 → 정제·전처리 → 지표·분모 → 표·차트 → 사실과 한계” 순서로 진행한다. `show_id`와 제목은 다르며, `date_added`와 `release_year`, 시청 등급과 사용자 평점, 작품 수와 장르 관계 수를 구별한다. 영화 5,377건(69.1%)이라는 결과나 상위 장르 관계 수는 **현재 스냅샷의 공급 구성**을 설명한다. 사용자 시청·선호·가입·수익·추천 효과는 이 파일만으로 평가할 수 없다. 결측이나 극단값을 발견했을 때 자동 삭제하지 않고, 분석 목적에 맞는 처리와 유효 N을 기록한다.